

User Acceptance Test case Generation using LLMs

Siddhesh Varpe

*Department of Computer Science and Engineering
Punjab Engineering College (Deemed to be University)
Chandigarh, India
0009-0005-5786-7515*

Uttam Mittal

*Department of Computer Science and Engineering
Punjab Engineering College (Deemed to be University)
Chandigarh, India
0009-0008-4177-6434*

Rajesh Kumar Bhatia

*Department of Computer Science and Engineering
Punjab Engineering College (Deemed to be University)
Chandigarh, India
rbhatia@pec.edu.in*

Ashpreet

*Department of Computer Science and Engineering
Punjab Engineering College (Deemed to be University)
Chandigarh, India
ashpreet@pec.edu.in*

Abstract—Understanding and validating user requirements is crucial to successful software development. However, manual User Acceptance Testing is often time-consuming, error-prone, and susceptible to bias, making automation an attractive alternative. This paper proposes a zero-shot learning framework for automating User Acceptance Testcase (UAT) generation using Large Language Models (LLMs). Five state-of-the-art LLMs—Gemma, LLaMA3, Mistral, Phi, and Qwen—are evaluated on the PURE dataset of Software Requirements Specification (SRS) documents. The methodology includes SRS text extraction, self-optimizing prompt generation, iterative test case generation, and comprehensive evaluation using both quantitative metrics (e.g., lexical similarity, semantic similarity via BERTScore, diversity, and test case volume) and qualitative analyses (e.g., word clouds and term frequency distributions). Results indicate that while Gemma produces the highest number of test cases, models like Qwen offer better semantic accuracy (with a BERTScore F1 of 80.58%), diversity, and broader requirement coverage. This work demonstrates the potential of LLMs to streamline UAT generation and highlights future directions for scaling and optimizing such frameworks within real-world software engineering workflows.

Index Terms—User Acceptance Testing, Software Requirements, LLMs, Zero-Shot Learning, Prompt Engineering, Test Case Automation, SRS Analysis

I. INTRODUCTION

Software development comprises several critical phases, each contributing to the delivery of a reliable and user-centric system. One of the most important phases is software testing, which occurs at multiple stages in the Software Development Life Cycle (SDLC), including unit testing, integration testing, system testing, and acceptance testing [1].

Acceptance testing, in particular, focuses on verifying whether the developed system meets the requirements outlined in the Software Requirement Specification (SRS) document. It serves as the final validation step to ensure that the delivered product aligns with the user's expectations. Despite its significance, acceptance testing—especially the creation of User Acceptance Test cases (UAT)—remains largely a manual process. This makes it time-consuming, labor-intensive, and prone to human errors. Furthermore, misinterpretation of requirements

during this stage can lead to costly delays in software delivery [2].

While significant progress has been made in automating unit and integration testing, acceptance testing remains largely manual. This is primarily due to the unstructured and narrative nature of SRS documents, in contrast to the structured representations—such as UML diagrams—often available for unit and integration testing.

Automating UAT generation can substantially improve the efficiency and accuracy of the testing process. Studies suggest that high-quality UATs can reduce software defects by over 30% through earlier detection of requirement mismatches [3]. Automation not only accelerates test case generation but also enhances test coverage, reduces ambiguity, and minimizes inter-observer bias.

Large Language Models (LLMs) have demonstrated strong capabilities in understanding and generating natural language, making them well-suited for interpreting SRS documents and generating corresponding UATs.

In this study, we propose a zero-shot learning framework that leverages the capabilities of LLMs to automatically generate UATs directly from SRS documents. Our method requires no prior labeled examples and operates without human intervention. We evaluate various LLM agents and employ prompt engineering strategies to optimize the relevance, completeness, and diversity of the generated test cases.

The rest of the study is structured as follows: Section II discusses the existing work and research that has been done in this field. Section III introduces the methodology used in the study. Section IV discusses the results obtained and compares them with the existing literature, and Section V concludes the study by summarizing the findings and the future scope of the study.

II. RELATED WORKS

Several approaches have been proposed to automate the generation of test cases that can be broadly classified into model-based studies and LLM-based studies.

A. Model based studies

Early efforts in this domain included utilising formal models or diagrams such as UML and State Machines. Kim et al. [4] propose using the UML diagrams for the testcase generation. Intana et al. [5] utilized the State Machine diagrams for performing the State transition-based testing. Their web-based framework parsed the XML data to extract the state trees as well as the test cases matrices, achieving an average Precision of 98.33%, a Recall of 97.68%, and an F1-score of 97.99%. Though effective, this approach is limited by the accuracy and the correct identification of the states and transitions in the State Machine Diagram provided as an input to the web framework. Moreover, state transition testing only focuses on individual state changes and transitions, missing out on the testing of various combinations or sequences of activities.

The author of [6] conducted a Systematic Mapping Study (SMS) on Model-Based Testing literature, reviewing 57 primary studies from the SMS and 77 SRS documents from a public repository. The PRO-TEST tool extracted 149 performance requirements and generated 96 test environments using a taxonomy tree. However, the method required extensive preprocessing, revealing 180 parameters that remained under-specified, limiting the automation of the process.

While effective, these methods mainly relied on formal methods or diagrams, which are generally unavailable in practical projects that use SRS documents written in natural language.

B. LLMs based studies

Recent advances in LLMs have opened up avenues for their application in generating UATs using SRS documents. Bhatia et al. [7] used a two-stage approach by first making the LLM summarize the SRS document, and then asking it to generate UATs for each of the use cases listed in the SRS. The authors used a dataset of 5 student SRS documents, generating about 87.7% valid test cases.

The authors of [8] collected a large dataset comprising of 168 projects, 1067 use cases, and 3606 test cases. They evaluated both fine-tuned as well as zero/one shot LLMs using BERT-based F1-scores for the semantic similarity. LLAMA-3.1 achieved the best results with an F1 of 98.94%, while Mistral achieved 90.4% and GPT-4o 88.43%, making it much closer to the human-written test cases.

Another study [9] examined the effectiveness of GPT-3.5 Turbo, which was used via the OpenAI API through the LangChain framework. Their semi-automated approach was evaluated by a team of 7 QA engineers on real-world web data (CESAR Dates Dataset). The AI-generated test cases outperformed humans on qualitative metrics, achieving an average score of 4.31/5 as compared to 4.18/5 for humans.

While these works demonstrate the viability of LLMs, most of them rely on supervised fine-tuning, a structured input format, or limited-domain datasets. Few explore a zero-shot generalization for UAT generation from an SRS document without domain adaptation or human intervention.

III. METHODOLOGY

This section details the steps used for generating the UATs from the SRS documents. Fig. 1 represents the complete pipeline of the proposed methodology. The pipeline processes the SRS documents, extracts requirements, generates UATs, and evaluates their quality through metrics and visualizations, minimizing manual effort and bias in requirement interpretation.

A. Dataset

For this study, we used Public Requirements (PURE) dataset collected from the Zenodo public repository [10]. The dataset comprises 79 publicly available natural language requirements documents collected from the Web. These documents vary in domain and complexity, covering multiple application areas. Each of these SRS is available in PDF format, having structured sections such as Introduction, Functional Requirements, Non-Functional Requirements, etc. The primary objective of this dataset in our study is to evaluate the effectiveness of large language models (LLMs) in the task of automated user acceptance test case generation from natural language software requirements. The dataset covers multiple domains but includes challenges like inconsistent formatting and ambiguous grammar, making it well-suited for evaluating the robustness and generalization of LLMs in our work.

B. PDF Text Extraction

Since LLMs are conventionally designed to process plain text and not files such as PDFs or images, all the relevant text needed to be extracted from the PDF documents of the dataset. Hence, each of the SRS documents present in the dataset was parsed using the PyPDF2 library of Python to extract all the text present in it. This library helps in efficiently extracting the text from it page-wise and then concatenating it to get the complete text from a PDF. The non-textual elements, such as some company logos or other figures that are usually irrelevant in the context of UAT generation, were removed during this phase to keep only the text.

C. LLMs evaluated

To extract relevant UATs from SRS documents, five LLM agents were evaluated and compared based on their generation capabilities, architecture diversity, and open-source accessibility. The selected models—Gemma, LLaMA, Mistral, Phi, and Qwen—represent a broad spectrum of high-performing open-weight language models. These LLMs were chosen for three primary reasons:

- strong benchmark performance across reasoning and instruction-following tasks,
- architectural variety encompassing transformer variants and tokenizer differences, and
- open-source availability, enabling reproducibility and cost-effective experimentation in academic settings.

This comparative analysis was aimed at identifying which open models are most suitable for structured, domain-specific

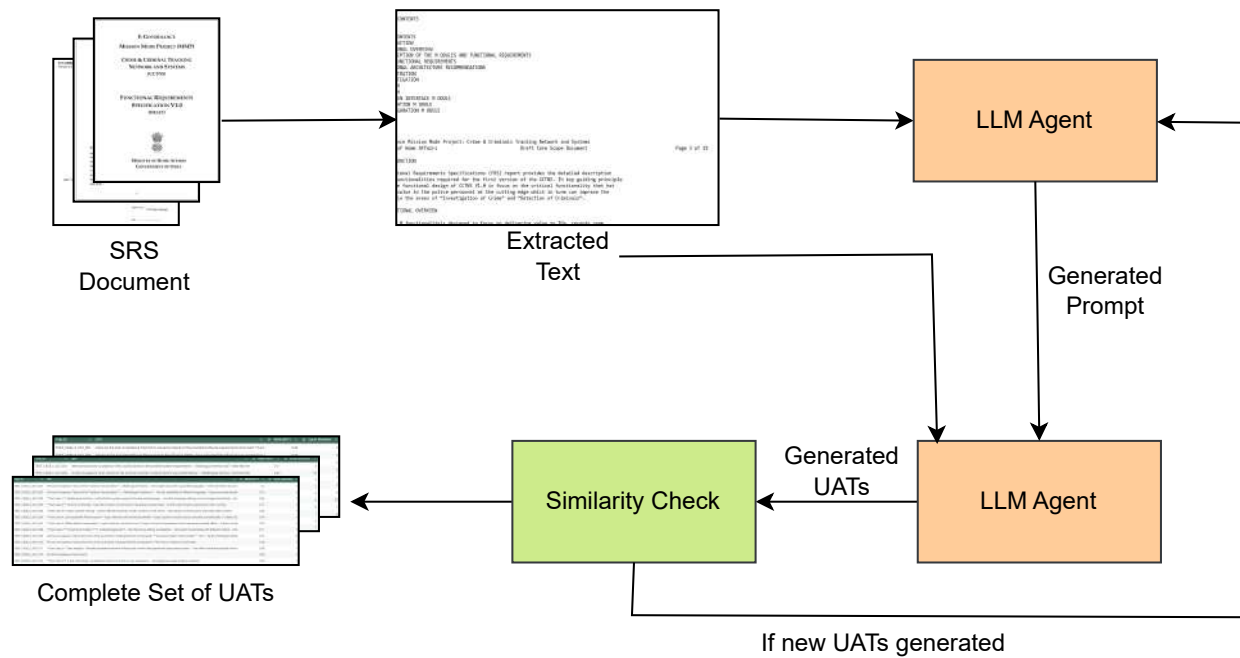


Fig. 1: Proposed methodology

generation tasks like UAT extraction, where precision, contextual understanding, and test case realism are essential.

D. Prompt Engineering

The UAT generation phase employs a **zero-shot learning framework** through a **two-stage prompt engineering strategy**. The extracted SRS text is first fed to the LLM agent along with a **base prompt** that instructs the model to analyze the SRS document and generate a **refined prompt** aimed at extracting key requirements and, eventually, relevant UATs. The base prompt used in this study is:

“Please analyze the following SRS document. Generate a clear, specific prompt that could be used to extract important requirements for generating User Acceptance Testcases.”

The refined prompt returned by the LLM agent is then used to generate UATs, ensuring context-specific and requirement-driven outputs. This **self-optimizing approach**, inspired by [8], improves output relevance and consistency.

E. Test Case Generation

Once the refined prompt was generated, it was provided to the same LLM agent (i.e., the one that generated the prompt) to produce the corresponding UATs. This approach was uniformly applied across all five LLMs evaluated in the study. For each model, test cases were generated iteratively over a maximum of 10 cycles, with each successive cycle leveraging the outputs from the previous one to maximize diversity and minimize redundancy in UAT generation.

To further control redundancy, an early stopping criterion with a patience value of 3 was implemented—i.e., if three consecutive cycles failed to yield any new unique UATs, the generation process was terminated early for that model. Duplicate or near-duplicate test cases were filtered using the `SequenceMatcher` utility, with a similarity threshold of 0.5 to ensure semantic variety.

Finally, a consolidated list of generated test cases was compiled for each LLM agent, aggregating all cycle-wise outputs along with metadata. This iterative, multi-model approach builds upon prior work demonstrating high test case validity in LLM-driven generation tasks [7].

F. Evaluation metrics

The visualization phase aims to evaluate the generated UATs. This is done by aggregating model-specific results, computing metrics such as mean/median similarity (via `SequenceMatcher`), unique cycle counts, and total unique UAT counts to assess the diversity and coverage, as recommended in [2]. Semantics were evaluated using `BERTScore` (Precision, Recall, and F1), comparing model-generated test cases against human-written ones. Visualizations were generated using `python` libraries such as `matplotlib`, `seaborn`, and `wordcloud`.

G. Experimental Setup

The pipeline is evaluated on the PURE dataset [10]. Five LLMs namely, Gemma, Mistral, Phi, LLaMA3, and Qwen [11]–[15] were tested via the `ollama` framework. All the models were run in Python 3.12 with `PyPDF2`, `ollama`, `pandas`,

and visualization libraries (matplotlib, seaborn, wordcloud). Experiments are conducted on a high-performance computing environment having an Intel Xeon Processor having 128 GB RAM and a high-end NVIDIA RTX 3090 Ti GPU with 24 GB VRAM.

This methodology leverages the zero-shot capabilities of LLMs to automate UAT generation, addressing challenges like manual effort and requirement misinterpretation. The proposed approach ensures robust processing, generation, and evaluation, as explored in section IV.

IV. RESULTS AND DISCUSSION

This study evaluates the proposed methodology for automating User Acceptance Test (UAT) case generation using Large Language Models (LLMs) on the PURE dataset. Five state-of-the-art open-weight LLMs—Gemma, LLaMA3, Mistral, Phi, and Qwen—were assessed within a zero-shot learning framework. The results, presented in Table I and Figures 2, 3, 4, and 5, provide comprehensive insights into the effectiveness, diversity, and quality of the UATs generated by each model. These findings form the basis for evaluating the viability of open-source LLMs in structured test case generation from SRS documents.

A. Model Performance Evaluation Metrics

Table I compares the performance of the five LLMs based on similarity metrics (mean, median, standard deviation), total test case counts, and an overall score combining diversity, unique cycle counts, and test case volume. Gemma generated the most test cases (1856) but had the highest mean similarity (0.418), indicating lower diversity due to redundant test cases. Mistral, with the lowest mean similarity (0.160) and 612 test cases, produced more diverse outputs. LLaMA3 balanced diversity and volume with a mean similarity of 0.219 and 764 test cases. Phi and Qwen generated fewer test cases (543 and 575, respectively) with moderate similarity scores (0.252 and 0.279), resulting in lower overall scores (0.68 for both).

The overall score, which prioritizes lower similarity (higher diversity), unique cycle counts, and test case volume, ranked Gemma highest (0.83), followed by LLaMA3 (0.76) and Mistral (0.74). This suggests that while Gemma produced the most test cases, Mistral and LLaMA3 were more effective at generating diverse test cases, aligning with the goal of maximizing requirement coverage.

TABLE I: Model Performance Comparison

Model	Similarity Mean	Similarity Median	Similarity Std	Test Cases	Score
Gemma	0.41	0.41	0.21	1856	0.83
Llama3	0.21	0.15	0.17	764	0.76
Mistral	0.16	0.13	0.08	612	0.74
Phi	0.25	0.26	0.13	543	0.68
Qwen	0.27	0.28	0.12	575	0.68

1) *Similarity and Diversity Analysis:* Figure 2 shows the similarity distribution of generated test cases. Mistral's low median similarity (0.130) and narrow range indicate consistent generation of unique test cases. Gemma's higher median

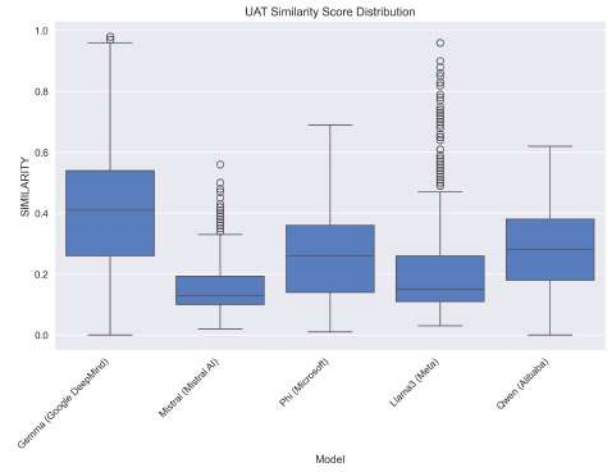


Fig. 2: Box plot Similarity Distribution

(0.410) and wider spread (standard deviation of 0.211) suggest it produced more similar test cases, possibly due to overfitting to certain SRS patterns. LLaMA3's low median similarity (0.150) and moderate spread (standard deviation of 0.173) indicate a strong balance, making it effective for diverse UAT generation.

The SequenceMatcher-based similarity threshold (0.5) effectively filtered duplicates, ensuring distinct test cases. This approach, inspired by prior work, enhanced diversity, particularly for Mistral and LLaMA3, which outperformed others in generating unique test cases across cycles.

2) *Test Case Generation Trends:* Figure 3 illustrates the average number of UAT cases generated per cycle. Gemma produced the highest number of test cases per cycle, peaking early but plateauing due to redundancy. Mistral and LLaMA3 showed steadier generation rates, with Mistral maintaining consistent, unique outputs. Phi and Qwen had lower generation rates, with diminishing returns after initial cycles, suggesting limitations in their zero-shot capabilities for complex SRS documents.

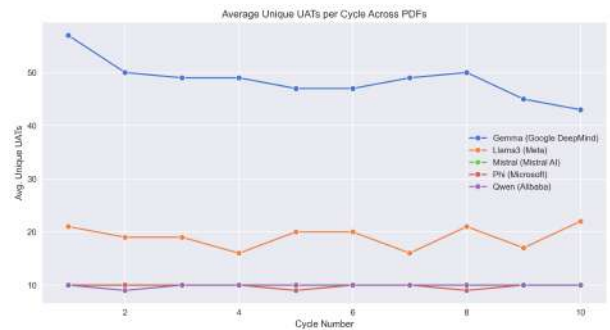


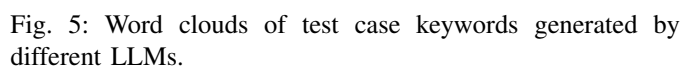
Fig. 3: Average UAT per Cycle (every LLM)

Figure 4 shows the number of UAT cases per SRS document. Gemma generated the most test cases across all SRS documents, but its high similarity scores indicate content overlap. Mistral and LLaMA3 produced fewer but more varied

Number of UATs per PDF for Each Model

PDF	genome	meta	gis	beta1	open
1	32	14	14	14	14
2	14	14	14	14	14
3	14	14	14	14	14
4	14	14	14	14	14
5	14	14	14	14	14
6	14	14	14	14	14
7	14	14	14	14	14
8	14	14	14	14	14
9	14	14	14	14	14
10	14	14	14	14	14
11	14	14	14	14	14
12	14	14	14	14	14
13	14	14	14	14	14
14	14	14	14	14	14
15	14	14	14	14	14
16	14	14	14	14	14
17	14	14	14	14	14
18	14	14	14	14	14
19	14	14	14	14	14
20	14	14	14	14	14
21	14	14	14	14	14
22	14	14	14	14	14
23	14	14	14	14	14
24	14	14	14	14	14
25	14	14	14	14	14
26	14	14	14	14	14
27	14	14	14	14	14
28	14	14	14	14	14
29	14	14	14	14	14
30	14	14	14	14	14
31	14	14	14	14	14
32	65	14	14	14	14
33	14	14	14	14	14
34	14	14	14	14	14
35	14	14	14	14	14
36	14	14	14	14	14
37	14	14	14	14	14
38	14	14	14	14	14
39	14	14	14	14	14
40	14	14	14	14	14
41	14	14	14	14	14
42	14	14	14	14	14
43	14	14	14	14	14
44	14	14	14	14	14
45	14	14	14	14	14
46	14	14	14	14	14
47	14	14	14	14	14
48	14	14	14	14	14
49	14	14	14	14	14
50	14	14	14	14	14
51	14	14	14	14	14
52	14	14	14	14	14
53	14	14	14	14	14
54	14	14	14	14	14
55	14	14	14	14	14
56	14	14	14	14	14
57	14	14	14	14	14
58	14	14	14	14	14
59	14	14	14	14	14
60	14	14	14	14	14
61	14	14	14	14	14
62	14	14	14	14	14

3) *Qualitative Insights:* Figure 5 highlights frequent terms in the generated test cases, such as “user”, “system”, “functionality”, and “requirement”, showing that LLMs effectively captured core SRS concepts. Variations in term frequency across models suggest differences in focus. Mistral emphasized “edge cases” and “error handling”, while Gemma’s outputs were more generic, focusing on “basic functionality”. This qualitative analysis underscores the importance of the two-stage prompt engineering strategy, which enhanced context-specific test case generation.



To evaluate the performance of the five Large Language Models (LLMs) throughout the prompt evolution process, using a generated prompt that evolves every cycle a framework was used. This framework checks for Qualitative and Quantitative insights to provide a review of each model’s capabilities. The qualitative evaluation was automated using an LLM-as-a-Judge approach; this methodology was chosen for its ability to approximate expert human judgment [16].

- **Clarity & Specificity:** This qualitative metric evaluates whether a UAT is unambiguous and contains concrete, actionable details. A high-scoring UAT provides specific examples and clear expected outcomes, whereas a low-scoring UAT is vague and resembles a general requirement.
- **Testability:** This qualitative metric assesses whether a non-technical user could reliably execute the test and determine a clear pass/fail outcome based solely on observable changes within the user interface. A high score indicates a true, user-centric UAT, while a low score suggests a test that requires technical backend knowledge.
- **Coverage:** This is the most critical quantitative metric, measuring the percentage of unique functional requirements from the source document for which at least one corresponding UAT was generated. It directly reflects the model's comprehension of the overall task.

The qualitative findings from the LLM-as-a-Judge analysis, summarized in Table II, add crucial context to the quantitative data. This analysis identified the single best-performing cycle for each model and reported its average qualitative scores.

LLM	Best Cycle	Clarity Score	Testability Score
Gemma	7	2.87	3.08
Llama3	6	3.13	3.49
Mistral	2	2.58	2.44
Phi	8	3.22	<u>3.38</u>
Owen	9	2.48	2.57

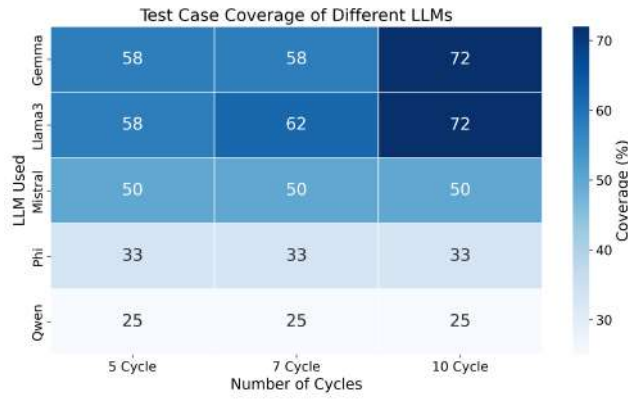


Fig. 6: Test case coverage over different LLMs across varied cycles

The results confirm Llama3’s superior capabilities. At its peak in Cycle (6/10), Llama3 achieved an average Clarity score of (3.13/5) and a Testability score of (3.49/5). In contrast, Gemma is 2nd best when all other factors taken in account, compared to the best cycle for Mistral at (2/10) but other scores bad, Phi with clarity score of (3.22/5) but delayed peak in cycle. This qualitative data provides strong evidence that Llama3 produced tests of a significantly higher practical quality, making them more valuable in a real-world software development context.

C. BERTScore-Based Evaluation

To evaluate the semantic similarity between generated test cases and provided requirements, we employed BERTScore metrics evaluation using manually written test cases for ground truth. Due to the effort involved in verifying all outputs, this evaluation was conducted on a single representative SRS document out of 79 SRS.

As shown in Table III, Qwen achieved the highest scores across all BERTScore metrics, reflecting strong semantic alignment with the ground truth cases. Gemma closely followed as the second-best performer, demonstrating its overall consistency and proving to be a reliable model for similar work. These results offer insight into the relative strengths of the models.

TABLE III: BERTScore Comparison

LLM Used	Precision	Recall	F1-Score
Gemma	0.7771	0.8268	0.8007
Llama3	0.7080	0.8107	0.7558
Mistral	0.7075	0.8181	0.7587
Phi	0.7471	0.8143	0.7787
Qwen	0.7851	0.8281	0.8058

D. Comparison with Prior Work

The authors of [7] used a two-stage LLM approach, using the first LLM to extract requirements from the SRS, and the second one to extract the UATs based on the requirements, thus generating 87.7% valid test cases. Though this is a modular

approach, it possesses the risks of contextual degradation and compounded error propagation. To mitigate these risks, the proposed study uses the first LLM’s prompt as a baseline, which itself is improved over the cycles, used to guide the second LLM, retaining its access to the complete SRS as well as enhancing the prompt, thus increasing the testcase coverage, as well as the quality of the testcases generated.

Another study reported high F1-scores (e.g., 89.94% for LLaMA3.1) with fine-tuned models, but in our zero-shot framework, Gemma and Qwen (score of greater than 80%) showed competitive performance without extensive training [8]. A state transition testing framework achieved a 97.99% F1-score but relied on structured diagrams, limiting its applicability to text-based SRS documents [5]. Moreover, the complexity of the approach increases overwhelmingly with the increase in the number of states and transitions in the system. Our methodology, using unstructured SRS text, offers broader applicability while maintaining high-quality outputs, independent of the number of states and transitions.

A qualitative evaluation in prior work noted AI-generated test cases slightly outperformed human-written ones (4.31/5 vs. 4.18/5) [9]. Our visualization phase, with box plots and word clouds, provides a more comprehensive evaluation, assessing both quantitative metrics and qualitative relevance, addressing correctness, consistency, and completeness.

E. Limitations and Challenges

The PURE dataset, while diverse, may not fully represent complex, industry-scale SRS documents, limiting generalizability. The zero-shot framework occasionally produced generic test cases, especially for Gemma, suggesting a need for dynamic prompt optimization or fine-tuning. Computational costs were significant, particularly for Gemma’s high-volume generation. Excluding non-textual SRS elements (e.g., diagrams) may have omitted critical requirements.

V. CONCLUSION & FUTURE SCOPE

This study leverages LLMs within a zero-shot learning framework to automate UAT generation from SRS. While the proposed approach demonstrates strong performance on the PURE dataset, several opportunities exist for further enhancement. Future work may focus on improving scalability by evaluating the methodology on larger and more diverse, industry-grade SRS datasets. The effectiveness of zero-shot prompting could be further optimized through dynamic strategies, such as reinforcement learning or few-shot adaptation, to enhance the contextual relevance and coverage of generated test cases. Also, the SRS documents can be parsed into XML files, which can help process information better using a structured input in contrast to unstructured PDF documents, thus enhancing the efficacy of the approach.

To address the computational overhead associated with iterative UAT generation, future implementations could explore parallelization or the use of lightweight LLM architectures for faster inference. Finally, integrating the proposed pipeline into CI/CD workflows and developing intuitive user interfaces

would help bridge the gap between academic research and practical industry adoption. These directions aim to improve the scalability, efficiency, and real-world applicability of automated UAT generation systems.

REFERENCES

- [1] G. J. Myers, C. Sandler, and T. Badgett, *The art of software testing*. John Wiley & Sons, 2011.
- [2] D. Damian and D. Zowghi, "Requirements engineering challenges in multi-site software development organizations," *Requirements Engineering Journal*, vol. 8, no. 1, pp. 149–160, 2003.
- [3] A. Edin and M. Qorbanzada, "An ambidextrous approach to user acceptance testing management and automation: How software customers can optimize their processes to explore new technology as well as current performance to improve the effectiveness of user acceptance testing," 2022.
- [4] Y. G. Kim, H. S. Hong, D.-H. Bae, and S. D. Cha, "Test cases generation from uml state diagrams," *IEEE Proceedings-Software*, vol. 146, no. 4, pp. 187–192, 1999.
- [5] A. Intana and A. Sawedsuthiphan, "Statetest: An automatic test case generation framework for state transition testing," in *2023 20th International Conference on Electrical Engineering/Electronics, Computer, Telecommunications and Information Technology (ECTI-CON)*. IEEE, 2023, pp. 1–4.
- [6] W. Abdeen, X. Chen, and M. Unterkalmsteiner, "An approach for performance requirements verification and test environments generation," *Requirements Engineering*, vol. 28, no. 1, pp. 117–144, 2023.
- [7] S. Bhatia, T. Gandhi, D. Kumar, and P. Jalote, "System test case design from requirements specifications: Insights and challenges of using chatgpt," *arXiv preprint arXiv:2412.03693*, 2024.
- [8] N. B. Hasan, M. A. Islam, J. Y. Khan, S. Senjik, and A. Iqbal, "Automatic high-level test case generation using large language models," *arXiv preprint arXiv:2503.17998*, 2025.
- [9] R. F. de Lima Junior, L. F. Paes de Barros Presta, L. Santos Borborema, V. Nogueira da Silva, M. Leal de Melo Dahia, and A. C. Sousa e Santos, "A case study on test case construction with large language models: Unveiling practical insights and challenges," *arXiv e-prints*, pp. arXiv–2312, 2023.
- [10] A. Ferrari, G. O. Spagnolo, and S. Gnesi, "Pure: A dataset of public requirements documents," in *2017 IEEE 25th international requirements engineering conference (RE)*. IEEE, 2017, pp. 502–505.
- [11] G. Team, T. Mesnard, C. Hardin, R. Dadashi, S. Bhupatiraju, S. Pathak, L. Sifre, M. Rivière, M. S. Kale, J. Love *et al.*, "Gemma: Open models based on gemini research and technology," *arXiv preprint arXiv:2403.08295*, 2024.
- [12] D. S. Chaplot, "Albert q. jiang, alexandre sablayrolles, arthur mensch, chris bamford, devendra singh chaplot, diego de las casas, florian bressand, gianna lengyel, guillaume lample, lucile saulnier, l  lio renard lavaud, marie-anne lachaux, pierre stock, teven le scao, thibaut lavril, thomas wang, timoth  e lacroix, william el sayed," *arXiv preprint arXiv:2310.06825*, 2023.
- [13] M. Abdin, J. Aneja, H. Awadalla, A. Awadallah, A. A. Awan, N. Bach, A. Bahree, A. Bakhtiari, J. Bao, H. Behl *et al.*, "Phi-3 technical report: A highly capable language model locally on your phone," *arXiv preprint arXiv:2404.14219*, 2024.
- [14] A. Grattafiori, A. Dubey, A. Jauhri, A. Pandey, A. Kadian, A. Al-Dahle, A. Letman, A. Mathur, A. Schelten, A. Vaughan *et al.*, "The llama 3 herd of models," *arXiv preprint arXiv:2407.21783*, 2024.
- [15] J. Bai, S. Bai, Y. Chu, Z. Cui, K. Dang, X. Deng, Y. Fan, W. Ge, Y. Han, F. Huang *et al.*, "Qwen technical report," *arXiv preprint arXiv:2309.16609*, 2023.
- [16] L. Zheng, W.-L. Chiang, Y. Sheng, S. Zhuang, Z. Wu, Y. Zhuang, Z. Lin, Z. Li, D. Li, E. Xing *et al.*, "Judging llm-as-a-judge with mt-bench and chatbot arena," *Advances in neural information processing systems*, vol. 36, pp. 46 595–46 623, 2023.
- [17] A. Singal and S. Goyal, "Comparative evaluation of ai platforms "google gemini 2.5 flash, google gemini 2.0 flash, deepseek v3 and chatgpt 4o" in solving multiple-choice questions from different subtopics of anatomy," *Surgical and Radiologic Anatomy*, vol. 47, no. 1, pp. 1–8, 2025.